

07. 핵심 알고리즘 (Core Algorithms)

07. 핵심 알고리즘 (Core Algorithms)

문서 정보

버전: v1.0 | 작성일: 2026-05-03 | 상태: 초안 | 작성자: 경북대학교 AI/빅데이터 전문가 양성 과정 — TBD팀

한 줄 요약

회사 가이드 정의 알고리즘 8개 + 우리가 직접 설계할 gap 영역 알고리즘 4개, 총 12개 알고리즘에 대해 텍스트 설명·Python 의사 코드·단위 테스트 케이스를 통합 제공한다. 회사 가이드 PPTX의 계산 예시를 단위 테스트로 활용해 “가이드와 동일한 결과”를 정량적으로 보장한다.

목차

- 1. 알고리즘 전체 지도
- 2. 공통 사양
- 3. 회사 정의 영역 — 표준 구현
 - 3.1 BMI 분류
 - 3.2 v1 — 권장 걸음수 + 기본점수
 - 3.3 v2 — 심박수 가중
 - 3.4 v3 — 백분위 보너스
 - 3.5 v4 — 만성질환 가중
 - 3.6 BMR (Mifflin-St Jeor)
 - 3.7 TDEE (활동계수 적용)
 - 3.8 7-step 체중 예측
- 4. Gap 영역 — 자체 설계 알고리즘
 - 4.1 영양제 OCR + LLM 파싱 파이프라인
 - 4.2 식단 → 영양소 변환
 - 4.3 부족 영양소 진단
 - 4.4 목적별 분석 매트릭스 (눈건강·간기능·피로회복)

- 5. 통합 파이프라인
- 6. 단위 테스트 전체 일람
- 7. 알고리즘 검증·고도화 전략

1. 알고리즘 전체 지도

● 회사 정의 영역 (구현만 하면 됨)

[활동점수]

- ① BMI 분류
- ② v1 (기본걸음점수)
- ③ v2 (심박 가중)
- ④ v3 (백분위 보너스)
- ⑤ v4 (만성질환 가중)

[영양 기준]

- ⑥ KDRIs 록업
- ⑦ BMI별 칼로리 조정

[체중 예측]

- ⑧ BMR Mifflin-St Jeor
- ⑨ TDEE
- ⑩ 7-step

● 갭 영역 (자체 설계 필요)

[입력 처리]

- ㉠ 영양제 OCR + LLM 파싱
- ㉡ 식단 → 영양소 변환

[분석·추천]

- ㉢ 부족 영양소 진단
- ㉣ 목적별 분석 매트릭스
(눈건강 / 간기능 / 피로회복)

2. 공통 사양

2.1 단위 표준

항목	단위	비고
체중	kg	소수점 1자리
키	cm	정수
나이	만 나이	정수
BMI	kg/m ²	소수점 1자리
칼로리	kcal	정수
영양소	mg / µg / g	KDRIs 표준
걸음수	보	정수
심박수	bpm	정수

2.2 입력 검증 (모든 알고리즘 공통)

```
def validate_user_inputs(
    age: int,
    sex: Literal["male", "female"],
    height_cm: float,
    weight_kg: float
) -> None:
    if not (1 <= age <= 120):
        raise ValueError(f"age must be 1-120, got {age}")
    if sex not in ("male", "female"):
        raise ValueError(f"sex must be male/female, got {sex}")
    if not (50 <= height_cm <= 250):
        raise ValueError(f"height_cm must be 50-250, got {height_cm}")
    if not (10 <= weight_kg <= 300):
        raise ValueError(f"weight_kg must be 10-300, got {weight_kg}")
```

3. 회사 정의 영역 — 표준 구현

3.1 BMI 분류

설명

한국·아시아 BMI 기준 (서양 25 미만 정상과 다름).

분류	BMI 범위	라벨
저체중	< 18.5	underweight
정상	18.5 ~ 22.9	normal
과체중	23.0 ~ 24.9	overweight
비만 1단계	25.0 ~ 29.9	obese_1
비만 2단계	≥ 30.0	obese_2

의사 코드

```

from enum import Enum
from dataclasses import dataclass

class BMICategory(str, Enum):
    UNDERWEIGHT = "underweight"
    NORMAL = "normal"
    OVERWEIGHT = "overweight"
    OBESE_1 = "obese_1"
    OBESE_2 = "obese_2"

def calculate_bmi(weight_kg: float, height_cm: float) -> float:
    """BMI = 체중(kg) / 키(m)²"""
    height_m = height_cm / 100
    return round(weight_kg / (height_m ** 2), 1)

def classify_bmi(bmi: float) -> BMICategory:
    """한국·아시아 BMI 분류"""
    if bmi < 18.5:
        return BMICategory.UNDERWEIGHT
    elif bmi < 23.0:
        return BMICategory.NORMAL
    elif bmi < 25.0:
        return BMICategory.OVERWEIGHT
    elif bmi < 30.0:
        return BMICategory.OBESE_1
    else:
        return BMICategory.OBESE_2

```

단위 테스트

```
import pytest

@pytest.mark.parametrize("weight,height,expected_bmi,expected_category", [
    (50.0, 170, 17.3, BMICategory.UNDERWEIGHT), # 저체중
    (60.0, 170, 20.8, BMICategory.NORMAL),      # 정상
    (70.0, 170, 24.2, BMICategory.OVERWEIGHT),   # 과체중
    (80.0, 170, 27.7, BMICategory.OBESE_1),     # 비만 1단계
    (90.0, 170, 31.1, BMICategory.OBESE_2),     # 비만 2단계
    # 경계값
    (53.4, 170, 18.5, BMICategory.NORMAL),      # 18.5 정확히
    (66.4, 170, 23.0, BMICategory.OVERWEIGHT),  # 23.0 정확히
])

def test_bmi_classification(weight, height, expected_bmi, expected_category):
    bmi = calculate_bmi(weight, height)
    assert bmi == pytest.approx(expected_bmi, abs=0.1)
    assert classify_bmi(bmi) == expected_category
```

3.2 v1 — 권장 걸음수 + 기본점수

설명

사용자별 권장 걸음수와 v1 기본 활동점수를 산출.

권장걸음수 = $8,000 \times \text{성별계수} \times \text{나이계수} \times \text{BMI계수}$

기본점수 = $\min(\text{실제걸음수} \div \text{권장걸음수}, 1.2) \times 83.33$

변수	조건	계수
성별계수	여성 / 남성	0.95 / 1.0
나이계수	40세 미만 / 40~59 / 60+	1.0 / 0.9 / 0.8
BMI계수	저체중 / 정상 / 과체중 / 비만1 / 비만2	0.9 / 1.0 / 1.05 / 1.1 / 1.15

의사 코드

```

def get_sex_factor(sex: str) -> float:
    return 0.95 if sex == "female" else 1.0

def get_age_factor(age: int) -> float:
    if age < 40:
        return 1.0
    elif age < 60:
        return 0.9
    else:
        return 0.8

BMI_FACTORS = {
    BMICategory.UNDERWEIGHT: 0.9,
    BMICategory.NORMAL: 1.0,
    BMICategory.OVERWEIGHT: 1.05,
    BMICategory.OBESE_1: 1.1,
    BMICategory.OBESE_2: 1.15,
}

def calculate_recommended_steps(
    sex: str, age: int, bmi_category: BMICategory
) -> int:
    """권장 걸음수 산출"""
    sex_f = get_sex_factor(sex)
    age_f = get_age_factor(age)
    bmi_f = BMI_FACTORS[bmi_category]
    return round(8000 * sex_f * age_f * bmi_f)

def calculate_v1_score(actual_steps: int, recommended_steps: int) -> float:
    """v1 기본점수 = min(달성률, 1.2) × 83.33"""
    achievement = min(actual_steps / recommended_steps, 1.2)
    return round(achievement * 83.33, 2)

```

단위 테스트 (회사 가이드 계산 예시 검증)

```
def test_v1_recommended_steps_50f_obese1():
    """50대 여성 비만1단계:  $8000 \times 0.95 \times 0.9 \times 1.1 = 7,524$ """
    steps = calculate_recommended_steps("female", 50, BMICategory.OBESE_1)
    assert steps == 7524

def test_v1_score_at_recommended():
    """달성률 100% → 83.33점"""
    score = calculate_v1_score(7524, 7524)
    assert score == pytest.approx(83.33, abs=0.01)

def test_v1_score_at_120_pct():
    """달성률 120% (상한) → 100점"""
    score = calculate_v1_score(9028, 7524)
    assert score == pytest.approx(100.0, abs=0.01)

def test_v1_score_above_120_capped():
    """달성률 200% → 여전히 100점 (상한)"""
    score = calculate_v1_score(15048, 7524)
    assert score == pytest.approx(100.0, abs=0.01)

def test_v1_50f_obese1_7000steps():
    """[가이드 예시] 7000보 / 7524 =  $0.93 \times 83.33 = 77.5$ """
    score = calculate_v1_score(7000, 7524)
    assert score == pytest.approx(77.5, abs=0.1)
```

3.3 v2 — 심박수 가중

설명

v1 기본점수에 심박수 강도를 곱해 운동의 질까지 반영.

목표심박구간 = $(220 - \text{나이}) \times [0.5 \sim 0.7]$
 심박계수 = $\min(\text{목표 심박 유지시간(분)} \div 30, 1.0)$
 ※ 웨어러블 미착용 시: 심박계수 = 0.7 (기본값)

 v2점수 = $v1점수 \times (0.7 + 0.3 \times \text{심박계수})$

의사 코드

```
def calculate_target_hr_range(age: int) -> tuple[int, int]:
    """목표 심박구간 (50%~70% of HRmax)"""
    hr_max = 220 - age
    return (round(hr_max * 0.5), round(hr_max * 0.7))

def calculate_hr_factor(
    target_hr_minutes: float | None
) -> float:
    """
    심박계수 = min(유지시간 / 30, 1.0)
    웨어러블 미착용 시 0.7 고정
    """
    if target_hr_minutes is None:
        return 0.7 # 기본값
    return min(target_hr_minutes / 30, 1.0)

def calculate_v2_score(v1_score: float, hr_factor: float) -> float:
    """v2 = v1 × (0.7 + 0.3 × 심박계수)"""
    multiplier = 0.7 + 0.3 * hr_factor
    return round(v1_score * multiplier, 2)
```

단위 테스트

```
def test_v2_target_hr_50yo():
    """50세 → HRmax 170 → 목표 85~119"""
    low, high = calculate_target_hr_range(50)
    assert (low, high) == (85, 119)

def test_v2_hr_factor_no_wearable():
    """웨어러블 없음 → 0.7"""
    assert calculate_hr_factor(None) == 0.7

def test_v2_hr_factor_under_30min():
    """20분 → 20/30 ≈ 0.667"""
    assert calculate_hr_factor(20) == pytest.approx(0.667, abs=0.01)

def test_v2_hr_factor_over_30min():
    """30분 이상 → 1.0 상한"""
    assert calculate_hr_factor(45) == 1.0

def test_v2_score_50f_example():
    """[가이드 예시] v1=77.5, 심박계수=0.667
    → v2 = 77.5 × (0.7 + 0.3 × 0.667) = 77.5 × 0.900 ≈ 69.7"""
    v2 = calculate_v2_score(77.5, 0.667)
    assert v2 == pytest.approx(69.7, abs=0.5)
```

3.4 v3 — 백분위 보너스

설명

같은 성별·연령대 그룹 내 사용자 순위에 따라 보너스 점수 부여.

비교 그룹 = 같은 성별 + 동일 연령대 (10세 단위)
 최소 표본 = 30명 이상

순위 보너스:

상위 10% 이내 → +10점
 상위 20% 이내 → +5점
 상위 30% 이내 → +3점
 그 외 → 0점

v3점수 = min(100, v2점수 + 백분위 보너스)

의사 코드

```
def calculate_percentile_bonus(
    user_v2: float, group_v2_scores: list[float]
) -> int:
    """
    그룹 내 백분위 → 보너스 점수
    표본 < 30명: 보너스 0
    """
    if len(group_v2_scores) < 30:
        return 0

    # 사용자보다 높은 점수의 비율 계산
    higher_count = sum(1 for s in group_v2_scores if s > user_v2)
    percentile_rank = (higher_count / len(group_v2_scores)) * 100
    # percentile_rank가 작을수록 상위

    if percentile_rank <= 10:
        return 10
    elif percentile_rank <= 20:
        return 5
    elif percentile_rank <= 30:
        return 3
    return 0

def calculate_v3_score(v2_score: float, bonus: int) -> float:
    """v3 = min(100, v2 + 보너스)"""
    return round(min(100.0, v2_score + bonus), 2)
```

단위 테스트

```
def test_v3_bonus_top_10pct():
    """상위 10% (5/100=5%) → +10"""
    group = [50.0] * 95 + [80.0] * 5
    user_v2 = 80.0
    bonus = calculate_percentile_bonus(user_v2, group)
    assert bonus == 10

def test_v3_bonus_below_threshold():
    """표본 < 30 → 0"""
    group = [50.0] * 20
    bonus = calculate_percentile_bonus(70.0, group)
    assert bonus == 0

def test_v3_score_capped_at_100():
    """v2=95 + 보너스 10 → 100 상한"""
    score = calculate_v3_score(95.0, 10)
    assert score == 100.0

def test_v3_50f_example():
    """[가이드 예시] v2=69.7, 상위 25% → +3 → v3=72.7"""
    v3 = calculate_v3_score(69.7, 3)
    assert v3 == pytest.approx(72.7, abs=0.1)
```

3.5 v4 — 만성질환 가중

설명

만성질환자가 동일 활동을 했을 때 더 높은 건강 효과를 누린다는 임상적 근거 반영.

만성질환가중 = $1.0 + \sum (\text{질환별 가산치})$ [최대 1.3 상한]

질환별 가산치:

당뇨	+0.10
고혈압	+0.10
심혈관질환	+0.15
관절질환	+0.15
호흡기질환	+0.10

v4점수 = $\min(100, \text{v3점수} \times \text{만성질환가중})$

의사 코드

```
DISEASE_WEIGHTS = {
    "diabetes": 0.10,
    "hypertension": 0.10,
    "cardiovascular": 0.15,
    "joint": 0.15,
    "respiratory": 0.10,
}
MAX_MULTIPLIER = 1.3

def calculate_disease_multiplier(diseases: list[str]) -> float:
    """만성질환 가중치 합산 (최대 1.3 상한)"""
    total_addon = sum(DISEASE_WEIGHTS.get(d, 0) for d in diseases)
    multiplier = min(1.0 + total_addon, MAX_MULTIPLIER)
    return round(multiplier, 3)

def calculate_v4_score(v3_score: float, multiplier: float) -> float:
    """v4 = min(100, v3 × multiplier)"""
    return round(min(100.0, v3_score * multiplier), 2)
```

단위 테스트

```
def test_v4_no_disease():
    assert calculate_disease_multiplier([]) == 1.0

def test_v4_single_disease():
    assert calculate_disease_multiplier(["diabetes"]) == 1.10

def test_v4_two_diseases():
    """[가이드 예시] 당뇨 + 고혈압 = 1.20"""
    assert calculate_disease_multiplier(["diabetes", "hypertension"]) == 1.20

def test_v4_capped_at_1_3():
    """4개 질환 → 합산 0.5이지만 상한 1.3"""
    diseases = ["diabetes", "hypertension", "cardiovascular", "joint"] # 0.5
    assert calculate_disease_multiplier(diseases) == 1.3

def test_v4_50f_example():
    """[가이드 예시] v3=72.7, 당뇨+고혈압=1.20
    → v4 = min(100, 72.7 × 1.20) = 87.2"""
    v4 = calculate_v4_score(72.7, 1.20)
    assert v4 == pytest.approx(87.2, abs=0.1)

def test_v4_unknown_disease_ignored():
    """미정의 질환은 무시"""
    assert calculate_disease_multiplier(["covid", "diabetes"]) == 1.10
```

3.6 BMR (Mifflin-St Jeor)

설명

기초대사량 (Basal Metabolic Rate). 완전 안정 상태에서의 하루 기본 소비 열량.

남성: $BMR = 10 \times W + 6.25 \times H - 5 \times A + 5$
 여성: $BMR = 10 \times W + 6.25 \times H - 5 \times A - 161$

W: 체중(kg) / H: 키(cm) / A: 나이(세)

의사 코드

```
def calculate_bmr(weight_kg: float, height_cm: float, age: int, sex: str) -> float:
    """Mifflin-St Jeor 공식"""
    base = 10 * weight_kg + 6.25 * height_cm - 5 * age
    constant = 5 if sex == "male" else -161
    return round(base + constant, 1)
```

단위 테스트

```
def test_bmr_50f_example():
    """[가이드 예시] 50세 여성 160cm 68kg
    BMR = 10×68 + 6.25×160 - 5×50 - 161 = 1,269"""
    bmr = calculate_bmr(68.0, 160, 50, "female")
    assert bmr == 1269.0

def test_bmr_45m_example():
    """[가이드 예시] 45세 남성 175cm 82kg
    BMR = 10×82 + 6.25×175 - 5×45 + 5 = 1,694"""
    bmr = calculate_bmr(82.0, 175, 45, "male")
    assert bmr == 1694.0

def test_bmr_male_female_diff():
    """동일 체격에서 남녀 BMR 차이 = 166 (5 - (-161))"""
    male_bmr = calculate_bmr(70, 170, 30, "male")
    female_bmr = calculate_bmr(70, 170, 30, "female")
    assert male_bmr - female_bmr == 166.0
```

3.7 TDEE (활동계수 적용)

설명

총 에너지 소비량 (Total Daily Energy Expenditure). 일일 걸음수를 활동 수준의 프록시로 사용.

$TDEE = BMR \times \text{활동계수}$

5,000보 미만	→ 1.200 (좌식 생활)
5,000 ~ 7,499보	→ 1.375 (가벼운 활동)
7,500 ~ 9,999보	→ 1.550 (보통 활동)
10,000 ~ 12,499보	→ 1.725 (활발한 활동)
12,500보 이상	→ 1.900 (매우 활발)

의사 코드

```
def get_activity_factor(daily_steps: int) -> float:
    """걸음수 → 활동계수"""
    if daily_steps < 5000:
        return 1.200
    elif daily_steps < 7500:
        return 1.375
    elif daily_steps < 10000:
        return 1.550
    elif daily_steps < 12500:
        return 1.725
    else:
        return 1.900

def calculate_tdee(bmr: float, daily_steps: int) -> float:
    """TDEE = BMR × 활동계수"""
    return round(bmr * get_activity_factor(daily_steps), 1)
```

단위 테스트

```
@pytest.mark.parametrize("steps, expected_factor", [
    (3000, 1.200),
    (6500, 1.375), # [가이드 예시] 6,500보 → 가벼운 활동
    (8000, 1.550), # [가이드 예시] 8,000보 → 보통 활동
    (11000, 1.725),
    (15000, 1.900),
    # 경계값
    (4999, 1.200),
    (5000, 1.375),
    (12500, 1.900),
])

def test_activity_factor(steps, expected_factor):
    assert get_activity_factor(steps) == expected_factor

def test_tdee_50f_example():
    """[가이드 예시] BMR 1,269 × 1.375 (6,500보) = 1,745"""
    tdee = calculate_tdee(1269.0, 6500)
    assert tdee == pytest.approx(1745.0, abs=0.5)
```

3.8 7-step 체중 예측

설명

N일 후 체중 예측. 핵심 원리: **체중 변화(kg) = 누적 에너지 수지(kcal) ÷ 7,700** (지방 1kg ≈ 7,700 kcal).

7단계

Step	내용
1	BMR 계산 (Mifflin-St Jeor)
2	$TDEE = BMR \times \text{활동계수}$
3	일일 수지 = 섭취칼로리 - TDEE
4	N일 누적 = $\sum(\text{일일 수지})$
5	이론 변화 = $\text{누적} \div 7,700$
6	현실 보정: 감량 $\times 0.85$ / 증량 $\times 0.95$
7	예측 체중 = 시작 체중 + 보정 변화

의사 코드

```

from dataclasses import dataclass

@dataclass
class WeightPrediction:
    bmr: float
    tdee: float
    daily_balance: float
    cumulative_balance: float
    theoretical_change: float
    corrected_change: float
    predicted_weight: float

KCAL_PER_KG_FAT = 7700
LOSS_CORRECTION = 0.85
GAIN_CORRECTION = 0.95

def predict_weight_n_days(
    weight_kg: float,
    height_cm: float,
    age: int,
    sex: str,
    daily_steps: int,
    daily_intake_kcal: float,
    n_days: int,
) -> WeightPrediction:
    """7-step 체중 예측"""
    # Step 1: BMR
    bmr = calculate_bmr(weight_kg, height_cm, age, sex)

    # Step 2: TDEE
    tdee = calculate_tdee(bmr, daily_steps)

    # Step 3: 일일 수지
    daily_balance = daily_intake_kcal - tdee

    # Step 4: N일 누적
    cumulative = daily_balance * n_days

    # Step 5: 이론 변화
    theoretical = cumulative / KCAL_PER_KG_FAT

    # Step 6: 현실 보정
    if daily_balance < 0:
        corrected = theoretical * LOSS_CORRECTION
    elif daily_balance > 0:
        corrected = theoretical * GAIN_CORRECTION
    else:
        corrected = theoretical

    # Step 7: 예측 체중
    predicted = weight_kg + corrected

    return WeightPrediction(
        bmr=bmr,
        tdee=tdee,
        daily_balance=round(daily_balance, 1),
        cumulative_balance=round(cumulative, 1),

```



```

        theoretical_change=round(theoretical, 3),
        corrected_change=round(corrected, 3),
        predicted_weight=round(predicted, 2),
    )

def predict_weight_periods(
    *args, **kwargs
) -> dict[int, WeightPrediction]:
    """1주 / 1개월(30일) / 3개월(90일) 예측 일괄"""
    return {
        days: predict_weight_n_days(*args, n_days=days, **kwargs)
        for days in (7, 30, 90)
    }

```

단위 테스트 (회사 가이드 계산 예시 그대로)

```

def test_weight_prediction_50f_30days():
    """[가이드 예시 1]
    50세 여성 160cm 68kg, 6,500보, 1,500kcal, 30일
    예상: 67.19kg"""
    pred = predict_weight_n_days(
        weight_kg=68.0, height_cm=160, age=50, sex="female",
        daily_steps=6500, daily_intake_kcal=1500, n_days=30
    )
    assert pred.bmr == 1269.0
    assert pred.tdee == pytest.approx(1745.0, abs=0.5)
    assert pred.daily_balance == pytest.approx(-245, abs=0.5)
    assert pred.cumulative_balance == pytest.approx(-7350, abs=2)
    assert pred.theoretical_change == pytest.approx(-0.955, abs=0.01)
    assert pred.corrected_change == pytest.approx(-0.81, abs=0.01)
    assert pred.predicted_weight == pytest.approx(67.19, abs=0.05)

def test_weight_prediction_45m_60days():
    """[가이드 예시 2]
    45세 남성 175cm 82kg, 8,000보, 2,231kcal (TDEE의 85%), 60일
    예상: 79.39kg"""
    pred = predict_weight_n_days(
        weight_kg=82.0, height_cm=175, age=45, sex="male",
        daily_steps=8000, daily_intake_kcal=2231, n_days=60
    )
    assert pred.bmr == 1694.0
    assert pred.tdee == pytest.approx(2625.0, abs=1)
    assert pred.predicted_weight == pytest.approx(79.39, abs=0.1)

def test_weight_maintenance():
    """섭취 = TDEE → 체중 유지"""
    pred = predict_weight_n_days(
        weight_kg=70.0, height_cm=170, age=30, sex="male",
        daily_steps=8000, daily_intake_kcal=0, n_days=30
    )
    # 일일 수지 = 0 - TDEE → 음수 → 감량
    assert pred.daily_balance < 0

```

4. 갭 영역 — 자체 설계 알고리즘

회사 가이드에 정의되지 않은 영역. 본 프로젝트의 진짜 도전.

4.1 영양제 OCR + LLM 파싱 파이프라인

입력·출력

- **Input:** 영양제 라벨 사진 (JPEG/PNG, 최대 5MB)
- **Output:** 구조화된 영양 성분 JSON

알고리즘 단계

1. 이미지 검증·전처리
 - 크기·포맷 확인
 - 회전·기울기 보정 (선택)
 - SHA-256 해시 → Redis 캐시 조회
2. OCR (Google Cloud Vision DOCUMENT_TEXT_DETECTION)
 - 라벨의 모든 텍스트 추출
3. LLM 구조화 (Claude API + Tool Use)
 - 비정형 텍스트 → 정형 JSON
4. 식약처 DB 매칭
 - 성분명 정규화 (예: "비타민 C" = "Vitamin C" = "ascorbic acid")
 - 식약처 건강기능식품 원료 DB 조회 → 기능성 정보 보강
5. 결과 캐싱 (Redis, TTL 30일)

의사 코드

```

from pydantic import BaseModel
from anthropic import Anthropic
from google.cloud import vision

class SupplementIngredient(BaseModel):
    name_ko: str          # 한국어 성분명
    name_en: str | None   # 영어 성분명
    amount: float
    unit: str             # mg, µg, g, IU
    daily_value_pct: float | None # %DV

class SupplementParseResult(BaseModel):
    product_name: str | None
    serving_size: str | None
    ingredients: list[SupplementIngredient]
    raw_ocr_text: str

async def parse_supplement_label(image_bytes: bytes) -> SupplementParseResult:
    # Step 1: 캐시 조회
    image_hash = hashlib.sha256(image_bytes).hexdigest()
    cached = await redis.get(f"ocr:{image_hash}")
    if cached:
        return SupplementParseResult.model_validate_json(cached)

    # Step 2: OCR
    vision_client = vision.ImageAnnotatorClient()
    response = vision_client.document_text_detection(
        image=vision.Image(content=image_bytes)
    )
    if response.error.message:
        raise OCRError(response.error.message)
    raw_text = response.full_text_annotation.text

    # Step 3: LLM 구조화
    claude = Anthropic()
    message = claude.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=2048,
        tools=[{
            "name": "extract_supplement_facts",
            "input_schema": SupplementParseResult.model_json_schema()
        }],
        tool_choice={"type": "tool", "name": "extract_supplement_facts"},
        messages=[{
            "role": "user",
            "content": f"""아래는 영양제 라벨의 OCR 결과입니다.
성분명·용량·단위·1일권장량 비율을 추출해 주세요.
한국어/영어 모두 지원하며, 한국어로 변환된 성분명을 우선합니다.

OCR 텍스트:
---
{raw_text}
---
"""
        }],
    )

```

```
# Tool use 결과 추출
tool_use_block = next(
    b for b in message.content if b.type == "tool_use"
)
parsed = SupplementParseResult.model_validate(tool_use_block.input)
parsed.raw_ocr_text = raw_text

# Step 4: 식약처 DB 매칭 (별도 함수)
parsed = await enrich_with_mfds_db(parsed)

# Step 5: 캐싱
await redis.set(
    f"ocr:{image_hash}",
    parsed.model_dump_json(),
    ex=60 * 60 * 24 * 30 # 30일
)

return parsed
```

단위 테스트

```

@pytest.mark.asyncio
async def test_parse_supplement_simple_label(mocked):
    """간단한 영양제 라벨 파싱"""
    mock_image = b"fake_image_bytes"

    # Cloud Vision 응답 모킹
    mock_vision = mocker.patch.object(vision_client, "document_text_detection")
    mock_vision.return_value = MagicMock(
        full_text_annotation=MagicMock(text="""
        Vitamin C 1000 mg 111% DV
        Vitamin D3 25 mcg 125% DV
        """),
        error=MagicMock(message="")
    )

    # Claude 응답 모킹
    mock_claude = mocker.patch.object(Anthropic, "messages")
    mock_claude.create.return_value = MagicMock(
        content=[MagicMock(
            type="tool_use",
            input={
                "product_name": None,
                "serving_size": None,
                "ingredients": [
                    {"name_ko": "비타민 C", "name_en": "Vitamin C",
                     "amount": 1000, "unit": "mg", "daily_value_pct": 111},
                    {"name_ko": "비타민 D3", "name_en": "Vitamin D3",
                     "amount": 25, "unit": "mcg", "daily_value_pct": 125},
                ]
            }
        )]
    )

    result = await parse_supplement_label(mock_image)
    assert len(result.ingredients) == 2
    assert result.ingredients[0].name_ko == "비타민 C"
    assert result.ingredients[0].amount == 1000

@pytest.mark.asyncio
async def test_parse_uses_cache_on_repeated_call(mocked):
    """동일 이미지 재호출 시 캐시 사용"""
    # ... (Redis 캐시 hit 검증)

```

4.2 식단 → 영양소 변환

Phase별 전략

Phase	입력	처리
Phase 2 (MVP)	텍스트 (예: “김치찌개 1그릇, 공깃밥 1그릇”)	식약처 식품영양성분 API 룩업
Phase 3 (고도화)	이미지	LLM Vision (Claude) 또는 AI Hub 한국 음식 모델

의사 코드 (Phase 2)

```
class FoodIntake(BaseModel):
    food_name: str
    amount: float
    unit: str # 그릇, 인분, g, ml

class NutritionalContent(BaseModel):
    energy_kcal: float
    carbs_g: float
    protein_g: float
    fat_g: float
    sodium_mg: float
    # ... 30종 영양소

async def parse_meal_text(meal_description: str) -> list[FoodIntake]:
    """LLM으로 텍스트 식단 → 구조화"""
    # "김치찌개 1그릇, 공깃밥 1그릇" →
    # [FoodIntake(food_name="김치찌개", amount=1, unit="그릇"), ...]
    ...

async def lookup_nutrition(food_name: str, amount: float, unit: str) -> NutritionalContent:
    """식약처 API에서 영양 정보 룩업 + 양 환산"""
    api_result = await mfd_api.search(food_name)
    serving_per_unit = api_result.servings.get(unit, 1.0)
    multiplier = amount * serving_per_unit
    return NutritionalContent(
        energy_kcal=api_result.energy * multiplier,
        carbs_g=api_result.carbs * multiplier,
        # ...
    )

async def calculate_meal_nutrition(meal_text: str) -> NutritionalContent:
    """식단 텍스트 → 총 영양소"""
    foods = await parse_meal_text(meal_text)
    contents = [
        await lookup_nutrition(f.food_name, f.amount, f.unit)
        for f in foods
    ]
    return sum_nutritional_contents(contents)
```

단위 테스트

```
@pytest.mark.asyncio
async def test_parse_simple_meal():
    """김치찌개 1그릇 → 1개 항목"""
    foods = await parse_meal_text("김치찌개 1그릇")
    assert len(foods) == 1
    assert foods[0].food_name == "김치찌개"
    assert foods[0].amount == 1
    assert foods[0].unit == "그릇"

@pytest.mark.asyncio
async def test_meal_nutrition_sum():
    """김치찌개 + 공기밥 영양소 합산"""
    nutrition = await calculate_meal_nutrition("김치찌개 1그릇, 공기밥 1그릇")
    assert nutrition.energy_kcal > 0
    assert nutrition.carbs_g > 0
```

4.3 부족 영양소 진단

알고리즘

1. 사용자 프로필 → KDRIs 록업 → 일일 권장 섭취량 (RDI) 산출
2. 식단 + 영양제 → 실제 섭취량 합산
3. 실제 / RDI 비율 계산
4. 비율 < 0.7 → 결핍 (deficient)
 비율 0.7~1.3 → 적정 (adequate)
 비율 > 1.3 → 과다 (excessive)
 비율 > UL/RDI → 위험 (risky)
5. 결핍 영양소 우선순위 정렬 (비율이 낮을수록 우선)
6. 만성질환자 분기 적용

의사 코드

```

from enum import Enum

class NutrientStatus(Enum):
    DEFICIENT = "deficient"
    LOW = "low"
    ADEQUATE = "adequate"
    EXCESSIVE = "excessive"
    RISKY = "risky"

class NutrientDiagnosis(BaseModel):
    nutrient_name: str
    rdi: float          # 권장 섭취량 (KDRIs)
    ul: float | None    # 상한 섭취량
    actual: float        # 실제 섭취량
    ratio: float         # actual / rdi
    status: NutrientStatus
    priority: int        # 결핍 우선순위 (낮을수록 시급)

DEFICIENT_THRESHOLD = 0.7
EXCESSIVE_THRESHOLD = 1.3

def diagnose_nutrients(
    user_profile: UserProfile,
    actual_intake: NutritionalContent,
) -> list[NutrientDiagnosis]:
    """30종 영양소 결핍 진단"""
    rdi_table = lookup_kdris(user_profile) # 사용자 맞춤 KDRIs

    diagnoses = []
    for nutrient_name in NUTRIENT_NAMES:
        rdi = rdi_table[nutrient_name]
        ul = rdi_table.get(f"{nutrient_name}_ul")
        actual = getattr(actual_intake, nutrient_name)
        ratio = actual / rdi if rdi > 0 else 0

        # 상태 분류
        if ul and actual > ul:
            status = NutrientStatus.RISKY
        elif ratio > EXCESSIVE_THRESHOLD:
            status = NutrientStatus.EXCESSIVE
        elif ratio < DEFICIENT_THRESHOLD * 0.5: # 35% 미만
            status = NutrientStatus.DEFICIENT
        elif ratio < DEFICIENT_THRESHOLD:
            status = NutrientStatus.LOW
        else:
            status = NutrientStatus.ADEQUATE

        diagnoses.append(NutrientDiagnosis(
            nutrient_name=nutrient_name,
            rdi=rdi, ul=ul, actual=actual,
            ratio=round(ratio, 2),
            status=status,
            priority=0, # 다음 단계에서 정렬
        ))

    # 결핍 우선순위 (비율이 낮을수록 우선)
    deficient = [d for d in diagnoses

```

```

        if d.status in (NutrientStatus.DEFICIENT, NutrientStatus.LOW)]
    deficient.sort(key=lambda d: d.ratio)
    for i, d in enumerate(deficient):
        d.priority = i + 1

    return diagnoses

```

단위 테스트

```

def test_diagnose_deficient_vitamin_c():
    """비타민C 권장 100mg, 실제 30mg → DEFICIENT"""
    profile = UserProfile(age=30, sex="male", ...)
    actual = NutritionalContent(vitamin_c_mg=30, ...)

    diagnoses = diagnose_nutrients(profile, actual)
    vit_c = next(d for d in diagnoses if d.nutrient_name == "vitamin_c_mg")

    assert vit_c.status == NutrientStatus.DEFICIENT
    assert vit_c.ratio == pytest.approx(0.30, abs=0.01)

def test_diagnose_risky_when_above_ul():
    """비타민A UL 3000 μg, 실제 5000 → RISKY"""
    profile = UserProfile(age=30, sex="male", ...)
    actual = NutritionalContent(vitamin_a_ug=5000, ...)

    diagnoses = diagnose_nutrients(profile, actual)
    vit_a = next(d for d in diagnoses if d.nutrient_name == "vitamin_a_ug")

    assert vit_a.status == NutrientStatus.RISKY

```

4.4 목적별 분석 매트릭스 (눈건강·간기능·피로회복)

의학적 근거 매트릭스

회사 가이드는 출력으로만 명시. 우리가 의학·식약처 기능성 인정 원료 기준으로 매트릭스 정의.

목적	핵심 영양소	KDRIs/식약처 권장량	근거
눈건강	루테인+지아잔틴	루테인 10~20 mg/일 (식약처)	AREDS2 — 황반변성 진행 억제
	오메가-3 (DHA)	1~2 g/일	망막 구성, 안구건조증
	비타민A	남 800 / 여 650 RAE	야맹증 예방
간기능	밀크씨슬 (실리마린)	≥130 mg/일	식약처 “간건강” 인정
	NAC (N-아세틸시스테인)	600~1,800 mg	글루타티온 전구체
피로회복	비타민B1 (티아민)	남 1.2 / 여 1.1 mg	에너지 대사
	비타민B2 (리보플라빈)	남 1.5 / 여 1.2 mg	FAD 구성
	비타민B12	2.4 µg	조혈
	코엔자임Q10	90~100 mg (건기식)	미토콘드리아
	마그네슘	남 350 / 여 280 mg	ATP 생성

⚠ 주의: 루테인은 흡연자에게 폐암 위험 가능성 → UI에 흡연자 분기 필요.

의사 코드

```

class HealthGoal(str, Enum):
    EYE_HEALTH = "eye_health"
    LIVER_FUNCTION = "liver_function"
    FATIGUE_RECOVERY = "fatigue_recovery"

GOAL_NUTRIENT_MATRIX = {
    HealthGoal.EYE_HEALTH: [
        ("lutein_zeaxanthin_mg", 10, 20, "AREDS2 황반변성 진행 억제"),
        ("omega3_dha_g", 1.0, 2.0, "망막 구성, 안구건조증"),
        ("vitamin_a_ug", None, None, "야맹증 예방 (KDRIs)"),
    ],
    HealthGoal.LIVER_FUNCTION: [
        ("milk_thistle_mg", 130, None, "식약처 간 건강 인정"),
        ("nac_mg", 600, 1800, "글루타티온 전구체"),
    ],
    HealthGoal.FATIGUE_RECOVERY: [
        ("vitamin_b1_mg", None, None, "에너지 대사 (KDRIs)"),
        ("vitamin_b2_mg", None, None, "산화적 인산화"),
        ("vitamin_b12_ug", None, None, "조혈"),
        ("coq10_mg", 90, 100, "미토콘드리아 전자전달계"),
        ("magnesium_mg", None, None, "ATP 생성 (KDRIs)"),
    ],
}

class GoalAnalysisResult(BaseModel):
    goal: HealthGoal
    score: float # 0~100
    recommendations: list[str]
    warnings: list[str]
    priority_nutrients: list[str]

def analyze_health_goal(
    goal: HealthGoal,
    user_profile: UserProfile,
    diagnoses: list[NutrientDiagnosis],
) -> GoalAnalysisResult:
    """목적별 영양소 충족도 분석"""
    matrix = GOAL_NUTRIENT_MATRIX[goal]
    relevant_diags = []
    warnings = []

    # 흡연자 + 눈건강 → 루테인 경고
    if goal == HealthGoal.EYE_HEALTH and user_profile.is_smoker:
        warnings.append(
            "흡연자는 루테인 고용량 보충 시 폐암 위험 가능성. "
            "전문가와 상담을 권장합니다."
        )

    # 매트릭스 영양소별 충족도 계산
    for nutrient_name, _, evidence in matrix:
        diag = next((d for d in diagnoses if d.nutrient_name == nutrient_name), None)
        if diag:
            relevant_diags.append(diag)

    if not relevant_diags:
        score = 0.0
    else:

```

```

# 평균 비율을 0~100으로 변환 (1.0이 만점)
avg_ratio = sum(min(d.ratio, 1.5) for d in relevant_diags) / len(relevant_diags)
score = round(min(avg_ratio, 1.0) * 100, 1)

# 권고: 결핍된 영양소 우선
deficient = [d for d in relevant_diags
             if d.status in (NutrientStatus.DEFICIENT, NutrientStatus.LOW)]
recommendations = [
    f"{d.nutrient_name} 보충 권장 (현재 {d.actual}, 권장 {d.rdi})"
    for d in deficient
]

return GoalAnalysisResult(
    goal=goal,
    score=score,
    recommendations=recommendations,
    warnings=warnings,
    priority_nutrients=[d.nutrient_name for d in deficient],
)

```

단위 테스트

```

def test_goal_eye_health_smoker_warning():
    """흡연자 + 눈건강 → 루테인 경고"""
    profile = UserProfile(age=45, sex="male", is_smoker=True, ...)
    diagnoses = [...]

    result = analyze_health_goal(HealthGoal.EYE_HEALTH, profile, diagnoses)
    assert any("흡연" in w for w in result.warnings)

def test_goal_score_when_all_deficient():
    """모든 핵심 영양소 결핍 → 낮은 점수"""
    profile = UserProfile(age=30, sex="female", ...)
    diagnoses = [
        NutrientDiagnosis(nutrient_name="lutein_zeaxanthin_mg",
                          rdi=15, actual=0, ratio=0,
                          status=NutrientStatus.DEFICIENT, ul=None, priority=1),
        # ...
    ]

    result = analyze_health_goal(HealthGoal.EYE_HEALTH, profile, diagnoses)
    assert result.score < 30

```

5. 통합 파이프라인

5종 출력을 한 번에 생성하는 통합 함수.

```

class FullAnalysisResult(BaseModel):
    deficient_nutrients: list[NutrientDiagnosis] # ① 부족 영양소
    recommended_intake: dict[str, float] # ② 권장 섭취량
    weight_predictions: dict[int, WeightPrediction] # ③ 1주/1개월/3개월
    activity_recommendation: ActivityRecommendation # ④ 운동 권고
    goal_analyses: dict[HealthGoal, GoalAnalysisResult] # ⑤ 목적별

async def run_full_analysis(
    user_profile: UserProfile,
    daily_steps: int,
    daily_intake_kcal: float,
    meal_text: str,
    supplement_images: list[bytes],
    target_hr_minutes: float | None = None,
    group_v2_scores: list[float] | None = None,
) -> FullAnalysisResult:
    """5종 출력 통합 생성"""
    # 1. 영양제 + 식단 → 영양소 합산
    sup_results = await asyncio.gather(*[
        parse_supplement_label(img) for img in supplement_images
    ])
    meal_nutrition = await calculate_meal_nutrition(meal_text)
    total_intake = sum_intake(meal_nutrition, sup_results)

    # 2. 부족 영양소 진단
    diagnoses = diagnose_nutrients(user_profile, total_intake)

    # 3. KDRIs 권장 섭취량
    rdi = lookup_kdris(user_profile)

    # 4. 체중 예측 (1주/1개월/3개월)
    predictions = predict_weight_periods(
        weight_kg=user_profile.weight_kg,
        height_cm=user_profile.height_cm,
        age=user_profile.age,
        sex=user_profile.sex,
        daily_steps=daily_steps,
        daily_intake_kcal=daily_intake_kcal,
    )

    # 5. 활동점수 v1~v4
    bmi = calculate_bmi(user_profile.weight_kg, user_profile.height_cm)
    bmi_cat = classify_bmi(bmi)
    rec_steps = calculate_recommended_steps(user_profile.sex, user_profile.age, bmi_cat)
    v1 = calculate_v1_score(daily_steps, rec_steps)

    hr_factor = calculate_hr_factor(target_hr_minutes)
    v2 = calculate_v2_score(v1, hr_factor)

    bonus = calculate_percentile_bonus(v2, group_v2_scores or [])
    v3 = calculate_v3_score(v2, bonus)

    multiplier = calculate_disease_multiplier(user_profile.diseases)
    v4 = calculate_v4_score(v3, multiplier)

    activity = ActivityRecommendation(
        recommended_steps=rec_steps,

```



```

        actual_steps=daily_steps,
        v1=v1, v2=v2, v3=v3, v4=v4,
    )

    # 6. 목적별 분석
    goal_results = {
        goal: analyze_health_goal(goal, user_profile, diagnoses)
        for goal in HealthGoal
    }

    return FullAnalysisResult(
        deficient_nutrients=[d for d in diagnoses
                             if d.status in (NutrientStatus.DEFICIENT, NutrientStatus.LOW)],
        recommended_intake=rdi,
        weight_predictions=predictions,
        activity_recommendation=activity,
        goal_analyses=goal_results,
    )

```

6. 단위 테스트 전체 일람

#	알고리즘	테스트 케이스 수	회사 가이드 검증
1	BMI 분류	7 (5 카테고리 + 2 경계값)	✓
2	v1 권장걸음수	5+	✓ (50대 여성 비만1)
3	v1 기본점수	4	✓ (7000보 사례)
4	v2 심박 가중	4	✓ (69.7 사례)
5	v3 백분위 보너스	4	✓
6	v4 만성질환	5	✓ (87.2 사례)
7	BMR	3+	✓ (1,269 / 1,694)
8	TDEE	7 (boundary)	✓ (1,745)
9	7-step 체중	3+	✓ (예시1, 예시2)
10	영양제 OCR 파싱	3+ (mocked)	—
11	식단 변환	3+	—
12	부족 영양소 진단	5+	—
13	목적별 분석	3+	—

총 단위 테스트 50+ 개, 회사 가이드 PPTX의 모든 계산 예시가 테스트로 자동 검증됨.

7. 알고리즘 검증·고도화 전략

7.1 검증 단계

단계	방법	통과 기준
레벨 1: 단위 테스트	pytest	50+ 테스트 100% 통과
레벨 2: 가이드 일치	회사 PPTX 계산 예시 재현	모든 예시값 ± 0.1 이내
레벨 3: 의료자문위 검토	영양사·의사 자문	표현·로직 검수 통과
레벨 4: 베타 테스트	30명 이상	사용자 피드백

7.2 한국인 특이값 보정 (Phase 3)

회사 가이드 BMR 공식(Mifflin-St Jeor)은 **미국·유럽 성인 기준**. 동일 BMI에서 체지방률이 서양인보다 높은 아시아인 특성으로 BMR이 5~10% 과대추정될 수 있음.

Phase 3 도입 옵션

옵션	입력 추가	효과
Katch-McArdle 공식	체지방률 (선택)	가장 정확 (체지방 측정 가능 시)
Cunningham 공식	제지방량 LBM	운동선수·근육질에 우수
한국인 보정 계수	없음	단순한 $\times 0.95$ 보정

7.3 Hall 동적 모델 (Phase 3 — 3개월 예측 정확도 향상)

7,700 kcal/kg 규칙은 단기(1~3개월) 예측에는 합리적이거나, 장기 예측엔 한계.

- Hall et al. 2011 (The Lancet) 동적 모델 도입 검토
- 적응성 열생성 + 체성분 변화 + BMR 변화 통합
- NIH Body Weight Planner 알고리즘 참조

권고 구현

예측 기간	방법
1주	$7,700 + \text{단기 보정} (\times 0.5 \sim 0.7) - \text{체수분 손실 보정}$
1개월	$7,700 + \text{회사 가이드 보정} (\times 0.85 \times 0.95)$
3개월	Hall 동적 모델

7.4 ML 적응형 보정 (Phase 3+)

사용자가 실제 측정한 체중을 학습 데이터로 활용: - 입력: (예측 체중, 실제 체중) 쌍 - 출력: 사용자별 보정 계수 (베이지안 업데이트) - 모델: Bayesian Linear Regression 또는 LSTM

 변경 이력

버전	날짜	변경 사항	작성자
v1.0	2026-05-03	초안 작성. 회사 정의 8개 + 갭 4개 알고리즘, 50+ 단위 테스트	TBD

 관련 문서

- [01. 프로젝트 개요](#)
- [06. 기술 스택](#)
- [08. 구현 계획](#)
- [09. 데이터·API 카탈로그](#)
- [10. 컴플라이언스 체크리스트](#)